



Lab-4.zip
Zip Archive

I have uploaded my complete Lab-4 Snake Game project.

Please inspect the project carefully and help me complete Attempt 1 of the assignment.

The assignment requires using an LLM as a debugging/pair-programming partner and completing the tasks iteratively within 3–4 attempts.

For Attempt 1, implement ONLY:

Show more ▾

Worked for 2m 5s >

Attempt 1 completed

I inspected:

- `main.py`
- `game/game_engine.py`
- `game/snake.py`
- `game/food.py`
- `README.md`

I made changes **only for Task 1 and Task 2**. Task 3 and Task 4 were not implemented.

Root causes

Task 1: `Snake.set_direction()` changed the direction immediately. Therefore, while moving RIGHT, pressing LEFT immediately changed the direction to LEFT, causing the head to move into the segment directly behind it.

Rapid presses created another problem because multiple direction changes could happen before the next actual grid movement.

Task 2: `game_over` was already set when a wall/self collision occurred, and `update()` stopped afterward, but `render()` only printed:

Game over! Final score: ...



There was no graphical Game Over screen or keyboard-controlled exit.

Changes made

game/snake.py

Added:

- next_direction
- turn_queued

The snake now **queues one valid turn** and applies it only when the snake actually moves.

For example:

Current: RIGHT
Press UP
Press LEFT
Next movement: UP



The LEFT press is ignored because one turn has already been queued.

A direct 180° turn is also rejected:

RIGHT → LEFT ✗
LEFT → RIGHT ✗
UP → DOWN ✗
DOWN → UP ✗



This preserves both arrow-key and WASD controls through the existing `GameEngine` mapping.

game/game_engine.py

Changed game-over handling so that:

- Wall collision still causes Game Over.
- Self collision still causes Game Over.
- Gameplay freezes after Game Over.
- Movement keys are ignored after Game Over.
- ENTER exits after Game Over.
- ESC exits after Game Over.

- A graphical **GAME OVER** screen appears.
- Final score is displayed.
- The existing board remains visible behind a dark overlay.

`main.py`

Changed the KEYDOWN handling so `GameEngine.handle_keydown()` can signal that the application should exit.

Therefore:

```
ENTER → exit
ESC   → exit
X     → exit normally
```



The normal window `QUIT` event remains unchanged.

Files modified

File	Purpose
<code>game/snake.py</code>	Fixed direction/reversal and rapid-key handling
<code>game/game_engine.py</code>	Added proper Game Over screen and post-game input handling
<code>main.py</code>	Added ENTER/ESC exit handling

Not modified: `game/food.py` , `README.md` , `requirements.txt` .

Manual test cases

You should perform these on your machine with Pygame installed.

Task 1

Test 1 — Opposite direction

1. Start the game.
2. Snake is moving RIGHT.
3. Immediately press LEFT.
4. Confirm the snake continues RIGHT instead of instantly reversing and dying.

Test 2 — Rapid key presses

1. Start while moving RIGHT.
2. Before the next movement, quickly press:
 - UP
 - LEFT
3. The next movement should be **UP**, not LEFT.
4. The snake should not die merely because LEFT was pressed afterward.

Test 3 — Normal turning

Test:

RIGHT → UP → LEFT → DOWN



Each turn should happen normally.

Test 4 — WASD

Test:

D → W → A → S



The controls should work the same way as the arrow keys.

Test 5 — Wall collision

Move into a wall.

Expected:

GAME OVER
Final Score: X
Press ENTER or ESC to exit



The snake should stop moving.

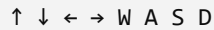
Test 6 — Self collision

Grow the snake by eating food and deliberately make it collide with itself.

Expected: the same Game Over screen and frozen gameplay.

Test 7 — Movement after Game Over

Once Game Over appears, press:



Nothing should move.

Test 8 — Exit

After Game Over:

- Press **ENTER** → application exits.
- Press **ESC** → application exits.
- Click **X** → application closes normally.

Verification status

I successfully performed:

- Python syntax checks on the modified files.
- Isolated Snake movement tests.
- Opposite-direction rejection test.
- Rapid **RIGHT → UP → LEFT** test.
- Normal subsequent turning test.

I **did not claim a full graphical Pygame test passed**, because Pygame is not installed in the execution environment and this environment cannot download it from the internet.

Attempt 1 project

[Download the completed Lab-4 Attempt 1 ZIP](#)

This ZIP contains the project with **Task 1 + Task 2 only**.



I implemented your Tasks 1 and 2 changes in my local project . I tested the game manually.

Test results:

- **Task 1 reverse-direction collision: WORKED**
- **Game Over screen and final score: WORKED**
- **ENTER/ESC exit: WORKED**
- **Arrow/WASD ignored after Game Over: WORKED**

Now proceed with **Attempt 2: Task 3 – Replay Option**.

Show more ▾

Worked for 36s >

Attempt 2 — Task 3 completed

I used your **Attempt 1** project as the **baseline** and made only the changes needed for **Task 3: Replay Option**.

What changed

Only `game/game_engine.py` needed modification.

`main.py` , `snake.py` , and `food.py` remain unchanged.

1. Replay menu after Game Over

The Game Over screen now shows:

GAME OVER
Final Score: X



Choose difficulty to replay

- 1 - Easy
- 2 - Medium
- 3 - Hard

ENTER/ESC - Exit

The final score remains visible.

2. Difficulty speeds

I defined:

Key	Difficulty	Speed
1	Easy	6 moves/sec
2	Medium	8 moves/sec
3	Hard	12 moves/sec

The existing Medium speed was 8 moves/sec, so the normal game speed is preserved.

3. Restart logic

Added:

</> Python



Run

When a difficulty is selected, it:

- Creates a fresh snake.
- Creates fresh food.
- Resets score to 0.
- Sets the selected speed.
- Resets the movement frame counter.
- Clears the Game Over state.

So selecting a difficulty starts a completely new game.

Functions changed

`GameEngine.__init__()`

- Added the Easy/Medium/Hard difficulty mapping.

`GameEngine.handle_keydown()`

- Added replay selection after Game Over.
- 1 → Easy
- 2 → Medium
- 3 → Hard
- ENTER/ESC → Exit
- Normal movement remains unaffected during gameplay.

`GameEngine.restart()`

- New function responsible for resetting the game.

`GameEngine.render()`

- Replaced the old exit-only Game Over message with the replay menu.

Task 4

Not implemented.

No sound effects, audio files, or sound-related functionality were added.

Verification

I performed:

- Python syntax compilation — **passed**
- Isolated replay-selection test — **passed**
- Verified selecting Hard resets score to 0
- Verified selecting Hard sets speed to 12
- Verified a new snake is created
- Verified ENTER/ESC still return the exit signal

I did **not** perform a full graphical Pygame run in this environment, so I am not claiming that manual graphical testing passed here.

Manual tests for you

Test 1 — Easy replay

1. Start the game.
2. Die by hitting a wall/self.
3. Confirm the Game Over screen appears.
4. Confirm the final score is visible.
5. Press 1.
6. Confirm a new game starts.
7. Confirm the snake moves slower than Medium.

Test 2 — Medium replay

1. Cause Game Over.
2. Press 2.
3. Confirm a new game starts.
4. Confirm the score starts from 0.
5. Confirm the speed is the original Medium speed.

Test 3 — Hard replay

1. Cause Game Over.
2. Press 3.
3. Confirm a new game starts.
4. Confirm the snake moves faster than Medium.

Test 4 — Score reset

For example:

Previous game:

Score = 8



Game Over

Press 1

New game:

Score = 0

The old final score should remain visible **until the replay is selected**.

Test 5 — Exit

At Game Over:

- Press **ENTER** → game exits.
- Press **ESC** → game exits.
- Click **X** → window closes normally.

Test 6 — Movement keys on Game Over

Press:

↑ ↓ ← →

W A S D



They should **not** restart or move the dead snake.

Attempt 2 project

[Download Lab-4 Attempt 2 ZIP](#)

This version contains **Tasks 1, 2, and 3**, with **Task 4** still untouched.



Attempt 2 is complete. I implemented the Task 3 replay option in my local project and tested it manually.

Test results:

- Game Over replay menu: WORKED
- Easy restart: WORKED
- Medium restart: WORKED

- Hard restart: WORKED
- Easy/Medium/Hard speed differences: WORKED
- Exit with 4: WORKED
- Exit with ESC: WORKED
- X button: WORKED
- Task 1 reverse-direction behavior after restart: WORKED
- Task 2 Game Over behavior: WORKED

Now proceed with Attempt 3: Task 4 – Sound.

Please inspect the current project and implement ONLY Task 4.

Requirements:

- Play a sound when the snake eats food.
- Play a different sound when Game Over occurs.
- Sounds must work during normal gameplay and when Game Over happens.
- Do not break Tasks 1, 2, or 3.
- Do not change the replay menu behavior.
- Do not unnecessarily modify unrelated code.
- Use pygame's existing audio functionality appropriately.
- If sound files are required, explain exactly where they should be placed and what filenames should be used.
- Make sure the game handles missing/unavailable sound files gracefully rather than crashing.
- Do not add any additional features outside Task 4.

Before making changes:

1. Inspect the current project.
2. Identify where food collision/eating occurs.
3. Identify where Game Over is triggered.
4. Identify the existing project structure and requirements.

After making changes:

1. Explain exactly which files changed.
2. Explain where the food-eaten sound is triggered.
3. Explain where the Game Over sound is triggered.
4. Explain where sound files should be stored.
5. Provide manual test cases for Task 4.
6. Do not claim a manual test passed unless it was actually tested.

Show less ^

Worked for 34s >

Attempt 3 — Task 4: Sound completed

I inspected the current Attempt 2 project and traced the existing flow before making changes.

Files changed

Only:

- `game/game_engine.py`
- Added `assets/sounds/eat.wav`
- Added `assets/sounds/game_over.wav`

No changes were made to:

- `main.py`
- `game/snake.py`
- `game/food.py`
- `requirements.txt`

So Tasks 1–3 and the replay menu behavior remain unchanged.

1. Food-eaten sound

Food collision already occurs in:

</> Python



▶ Run

Specifically, after movement and collision checks:

</> Python



▶ Run

I added the sound immediately after the food is consumed and respawned:

```
self.snake.grow()
self.score += 1
self.food.respawn(self.snake.body)
```

</> Python



Run

Therefore the sound plays once whenever the snake successfully eats food.

2. Game Over sound

Previously, both wall and self collision directly did:

</> Python



Run

I replaced those with:

</> Python



Run

The new function is:

```
def _trigger_game_over(self):  
    if self.game_over:  
        return  
    self.game_over = True  
    if self.game_over_sound is not None:  
        self.game_over_sound.play()
```

This means the Game Over sound is triggered exactly when the game transitions into the Game Over state.

It also prevents the sound from being repeatedly played if Game Over is already active.

3. Sound files

The project now contains:

Lab-4/

```
├── assets/
│   ├── sounds/
│   │   ├── eat.wav
│   │   └── game_over.wav
├── game/
│   ├── game_engine.py
│   ├── snake.py
│   └── food.py
└── main.py
```



I included working WAV files in the project, so you don't need to create or download them separately.

4. Missing sound handling

The sound initialization is protected with error handling.

If:

- a sound file is missing,
- Pygame's mixer is unavailable, or
- a sound cannot be loaded,

the game simply sets the sound to `None` and continues running.

So a missing/unavailable sound **will not crash the game**.

Verification performed

I verified:

- All modified Python files compile successfully.
- `eat.wav` is a valid WAV file.
- `game_over.wav` is a valid WAV file.
- The sound files have the expected PCM audio structure.
- Food sound is placed only in the food-consumption path.
- Game Over sound is placed only in the Game Over transition.
- Replay logic was not modified.

I did not claim that the actual audio playback passed, because this execution environment does not have a usable Pygame audio runtime. You should perform the following manual tests locally.

Manual Task 4 tests

Test 1 — Food sound

1. Start the game.
2. Move the snake onto food.
3. Listen for the food-eaten sound.
4. Eat another food item.
5. Confirm the sound plays again.

Test 2 — Wall Game Over sound

1. Start the game.
2. Move directly into a wall.
3. Confirm the Game Over screen appears.
4. Confirm the Game Over sound plays.

Test 3 — Self-collision sound

1. Grow the snake by eating several foods.
2. Make the snake collide with itself.
3. Confirm the Game Over screen appears.
4. Confirm the Game Over sound plays.

Test 4 — Sound + replay

1. Cause Game Over.
2. Confirm the Game Over sound plays.
3. Select **1**, **2**, or **3**.
4. Confirm the replay starts normally.
5. Eat food again.
6. Confirm the food sound still works.

Test 5 — No repeated Game Over sound

1. Cause Game Over.
2. Keep waiting on the Game Over screen.
3. Press movement keys.
4. Confirm the Game Over sound does **not** continuously replay.

Test 6 — Missing sound file

Temporarily rename one of the WAV files, for example:

`eat.wav → eat_backup.wav`



Start the game and eat food.

Expected: **the game continues without crashing**; only the sound is unavailable.

Attempt 3 project

[Download Lab-4 Attempt 3 ZIP](#)

This version contains **Tasks 1, 2, 3, and 4**, with no additional features added.



ChatGPT can make mistakes. Check important info.